

ThreadLocal class

18 April 2026 18:23

ThreadLocal class provides thread local variables. ThreadLocal class maintain value per thread basis. Each thread local object maintains a separate value like user_ID, transaction_ID etc. for each thread that access that object.

Thread can access its local value, can manipulate its and even can remove its value. In every part of the code which is executed by the thread, we can access its local variable.

Consider an example a servlet which invokes some business methods. We have a requirement to generate a unique transaction_ID each and every request and we have to pass this transaction_ID to the business methods, for this requirement we can use ThreadLocal to maintain a separate transaction_ID for every request, i.e., for every thread.

Note:

1. Local class introduced in 1.2 version and enhanced in 1.5 version.
2. ThreadLocal can be associated with ThreadScope.
3. Total code which is executed by the thread has access to corresponding thread local variables.
4. A thread can access its own local variables and can't access other threads local variables.
5. Once thread. Entered into dead state. All its local variable are by default eligible for garbage collection.

How to create ThreadLocal?

```
ThreadLocal tl = new ThreadLocal(); // Creates a thread local variable or object.
```

Methods:

1. Object get(): Returns the value of ThreadLocal variable associated with current thread.
2. Object initialValue(): Returns initial value of ThreadLocal variable association with the current thread. The default implementation of this method returns null. To customize our own initial value, we have to override this method.
3. void set(Object newValue): To set a new value.
4. void remove(): To remove the value of a ThreadLocal variable associated with current thread. (added in version 1.5.) After removal, if we try to access it, it will be reinitialized once again by invoking the initialValue() method.

```
package MyThreads.MultithreadingEnhancement;
public class ThreadLocalDemo {
    public static void main(String[] args) {
        ThreadLocal tl = new ThreadLocal<>();
        System.out.println(tl.get());
        tl.set("Shiv");
        System.out.println(tl.get()); //Shiv
        tl.remove();
        System.out.println(tl.get()); //null
    }
}
```

```
}  
}
```

Output:

```
null  
Shiv  
null
```

```
package MyThreads.MultithreadingEnhancement;  
public class ThreadLocalDemo2 {  
    public static void main(String[] args) {  
        ThreadLocal t1 = new ThreadLocal<>(){  
            @Override  
            protected Object initialValue() {  
                return "abc";  
            }  
        };  
        System.out.println(t1.get()); //abc  
        t1.set("Durga");  
        System.out.println(t1.get()); //Durga  
        t1.remove();  
        System.out.println(t1.get()); //abc  
    }  
}
```

Output:

```
abc  
Durga  
abc
```

```
package MyThreads.MultithreadingEnhancement;  
public class CustomerThread extends Thread {  
    static Integer custId = 0;  
    private static ThreadLocal t1 = new ThreadLocal<>(){  
        protected Integer initialValue() {  
            return ++custId;  
        }  
    };  
};  
  
CustomerThread(String name){  
    super(name);  
}  
@Override  
public void run() {  
    System.out.println(Thread.currentThread().getName()+  
        " executing with customer id: " + t1.get());  
}  
}
```

```

package MyThreads.MultithreadingEnhancement;
public class CustomerThreadApplied {
    public static void main(String[] args) {
        CustomerThread c1 = new CustomerThread("Arjun");
        CustomerThread c2 = new CustomerThread("Nakul");
        CustomerThread c3 = new CustomerThread("Yudhistir");
        CustomerThread c4 = new CustomerThread("Bhim");
        CustomerThread c5 = new CustomerThread("Shadev");
        c1.start();
        c2.start();
        c3.start();
        c4.start();
        c5.start();
    }
}

```

Output:

```

Nakul executing with customer id: 2
Arjun executing with customer id: 1
Yudhistir executing with customer id: 3
Bhim executing with customer id: 4
Shadev executing with customer id: 5

```

In the above program, for every customer thread, a separate customer ID will be maintained by the ThreadLocal object.

ThreadLocal with respect to inheritance:
What is the behavior of ThreadLocal?

Parent thread ThreadLocal variable by default not available to the child thread. If we want to make parent thread. ThreadLocal variable value available to child thread then we should go for **InheritableThreadLocal** class.

By default a child thread value is exactly same as parent thread value. But we can provide customized value for child thread by overriding the **Object childValue (Object O)** method.

Constructor:

```

InheritableThreadLocal itl = new InheritableThreadLocal();

```

Methods: InheritableThreadLocal is a child class of ThreadLocal class and hence all method present in parent class by default available in InheritableThreadLocal class. In addition to these methods, it contains only one method:

Modifier and Type	Method	Description
protected T	childValue(T parentValue)	Computes the child's initial value for this inheritable thread-local variable as a function of the parent's value at the time the child thread is created.

```

package MyThreads.MultithreadingEnhancement;

```

```

public class ParentThread extends Thread{
    public static InheritableThreadLocal itl = new InheritableThreadLocal<>(){
        protected Object childValue(Object parentValue) {
            return "Child value";
        }
    };
    @Override
    public void run() {
        itl.set("Parent value");
        System.out.println("Parent thread value---> " + itl.get());
        ChildThread ct = new ChildThread();
        ct.start();
    }
}

```

```

package MyThreads.MultithreadingEnhancement;
public class ChildThread extends Thread{
    @Override
    public void run() {
        System.out.println("Child Thread Value ---> " +
            ParentThread.itl.get());
    }
}

```

```

package MyThreads.MultithreadingEnhancement;
public class InheritableThreadLocalDemo {
    public static void main(String[] args) {
        ParentThread pt = new ParentThread();
        pt.start();
    }
}

```

Output:

```

Parent thread value---> Parent value
Child Thread Value ---> Child value

```

In the above program, if we replace. Inheritable thread local with thread local. And if we are not overriding child value () method. The output will be:

```

Parent thread value---> Parent value
Child Thread Value ---> null

```

Change in the code is :

```

package MyThreads.MultithreadingEnhancement;
public class ParentThread extends Thread{
    // public static InheritableThreadLocal itl = new InheritableThreadLocal<>(){
    //     protected Object childValue(Object parentValue) {
    //         return "Child value";
    //     };
    // };
    public static ThreadLocal itl = new ThreadLocal<>();
    @Override

```

```

    public void run() {
        itl.set("Parent value");
        System.out.println("Parent thread value---> " + itl.get());
        ChildThread ct = new ChildThread();
        ct.start();
    }
}

```

What is ThreadLocal?

ThreadLocal provides thread-specific variables.

Each thread accessing a ThreadLocal variable gets its own independent copy of that variable.

So:

- Threads do not share the same value.
- Each thread has its own isolated value.

Why do we need ThreadLocal?

In multithreading:

- Shared variables -> can use race conditions
- Synchronization -> adds overhead & complexity

ThreadLocal avoids both by:

- Eliminating sharing
- Avoiding Synchronization

When to use ThreadLocal?

Use it when:

- You want per-thread data
- Data should not be shared
- Example:
 - User sessions
 - Database connections
 - Logging context
 - Date formatters

ThreadLocal vs Synchronization

Feature	ThreadLocal	Synchronization
Data Sharing	No	Yes
Performance	Faster	Slower
Complexity	Low	High
Safety	High	High

Important: ThreadLocal is not for sharing data, it's for isolating data per thread.